ECM2414 Software Development Coursework Report

Student 1: Cheuk Yin Yew

Student 2: Dawid Galazkiewicz

1. Architecture and Object-Orientated Design

Card class has values of the cards. CardDeck class organizes the order of the cards. Player class manages hands and individual actions. CardGame class works on the setup and execution of the game. Splitting the program into 4 classes allowed us to test behavior individually, without having to execute 1 big program.

Card: Stores its values in a private, final integer field. Its constructor rejects negative values, while we have a getter method so that the values can still be read in other methods or classes. After construction, the cards are unchanged, which is important for security reasons and for when they are changing between threads. We have also tested edge cases for example values like 0 or values that are above the current player count, and we made sure that they are kept.

CardDeck: Uses an ArrayDeque that implements the Deque<Card> interface. Drawing removes the first card and discarding appends to the end. This retains the necessary first-in, first-out feature that is required by the coursework spec. The deck's internal collection is private, and getContents() method returns a copy of the collection, this prevents callers from changing the queue through that method, making our class more secure and preventing unintentional changing of the queue order.

Player: Has a copy of the four-card starting hand and has its own input deck, output deck and shared game lock. Its hand getter returns the current hand and it is unchangeable from that method. During each turn, it will draw one card and discard the oldest card where the value differs from its player's number. This means that we can keep preferred cards without keeping all of the unwanted cards across that player's turns that are successful.

Winning is checked separately: any four equal card values win, this also includes values that could differ from the player's number. For example if at the start of the game a player, had 4 values that were the same but not equal to its number then they should win. Or, very rarely if they don't receive their own player number and somehow discard all numbers that are different resulting in 4 same values for a 4 player game. However these events would be rare but we wanted for edge cases in the program to be satisfied.

CardGame: Constructs the ring of players cards and decks as well as, dealing cards in a round-robin fashion. For zero-based player index i, the output deck is (i+1) % n. This connects and deals with the single-player case to one shared input/output deck. Thread coordination remains and is contained with this class rather than giving Player class another game loop.

2. Thread co-ordination and trade-offs

Our game creates one Java thread per player. All connected players and decks will receive the same gameLock object. Synchronization protects the complete draw/discard operation, the winner state and co-ordination flags. Other player threads don't have the functionality to observe the temporary five-card hand or alter shared decks halfway through a turn. The declaration of a winner works in the same way, allowing only the first successful declaration to change the winner from zero.

We make sure the starting hands are checked before playing. Each worker thread increments a shared counter after its check. If no winner exists, it waits until all starting hands have been examined. This prevents a player from taking a turn before another player's initial winning hand is recognized. Once a winner exists, workers leave without any further moves.

We have an empty input deck that makes the method takeTurn() return false without changing any cards. The worker thread then calls the wait() method that releases the Thread rather than constantly polling all the time. Successful turns will use notifyAll() method due to having a discard that might enable another player to continue. After the thread wakes again a Worker Thread rechecks the stopping conditions and attempts the operation again.

The main thread uses join() outside the game monitor to wait for workers to finish. Only afterwards the monitor of the game, to wait for the Worker Threads to finish. Only afterwards does it write final player messages and deck contents, avoiding outputting snapshots to be taken during turns that haven't finished. Worker Thread runtime failures are recorded, set using the stopping flag and wake awaiting Worker Threads. if the main thread is interrupted whilst joining, it requests stopping, which consequentially interrupts the workers, waits for them and restores its interrupt status before reporting interruption.

The Monitor that is shared allows us to simplify consistency throughout, but it serialises complete turns across the whole game. Separate player threads therefore won't perform independent turns simultaneously, and this monitor provides no guarantee of equal playing opportunities. Logging also occurs while holding this lock, so slow file operations delay every player. These are quite significant limitations, especially if we consider assessing the requirements for independent player progress.

Something else we could have used in our design of the program is to have separate deck locks, with a consistent acquiring order so that we can avoid circular waiting, as well as separating the coordination as to decide which player is the winner. This could be a potential redesign that we could implement, however due to the time constraints and our program still has the required functionality; it is something that we decided not to implement.

3. Behavior of our program and Input Validation

The game repeatedly requests a positive integer player count and a readable pack containing exactly 8 * n lines per coursework requirement. Every line must parse as a non-negative Java integer. Invalid input produces a message and provides another attempt. Player construction also rejects invalid starting hands and inconsistent deck connections.

Initial player logs are written before workers start. Each successful turn records the drawn card, discarded card and current hand. After the Worker Threads finish, player logs record the result and final hand, whilst deck files contain the final queue order. Files for the same player/deck numbers are replaced in future games.

4. Testing Strategy and evidence

We have used JUnit Jupiter 5.14.4. We ran a test on 27 October 2026, using the Eclipse Temurin JDK 21.0.12.1+1. Our program has managed to pass all the 12 test methods that we wrote, without any failures or skipping any of the tests. Some of our methods test multiple inputs. However JUnit reports 12 tests rather than a separate result for every assertion or loop, as to make it more concise.

We have 12 methods for testing @Test, that are implemented using JUnit. Check the readme.md for more information on these.

Description of the tests:

Unit Tests call public operations on fresh objects. Player conservation checks count the actual card objects across the hand and both decks, detecting missing or duplicated cards even when denominations do repeat. Integration checks compare the denomination frequencies across all final hands and decks against the original pack.

The game tests launch the public entry point in a separate Java process, with packs that have been generated and temporary working directories. This isolates input, output files and threads from the JUnit process. A ten-second limit detects failure to terminate for the chosen processes. Tests use one, two and four players where appropriate, including the one-deck ring and an immediate win with no moves.

The normal-game tests will accept any winner that comes first rather than relying on the order of the schedule, this means that the first player thread that has a winning hand will win the game rather than relying on the dealing sequence. The output tests check all of the different player's hand changes and they check each deck's sequence of cards being put in and put out from the initial contents as well as the different draws and incoming discards.

Development Process

The GitHub repository records the process of implementing the deck model, player actions, game coordination, logging and automated tests.

Work distribution

Student 1 is responsible for implementing the CardGame and CardDeck class. The implementation includes validation, pack loading, round-robin dealing behaviour, deck operations and how the threads work together. Student 2 worked on the Card and Player class, which includes card representation, hand management, winning-hand checks, drawing and discarding hands, and player logging.

For the test suite, the work is divided into testing the game behavior part and the individual components. Student 1 is responsible for the CardGameTest. Student 2 is responsible for the CardTest, CardDeckTest and PlayerTest.

The README is split equally by both students. It documents the project structure, software requirements, game rules and the instructions for compiling, running and testing the application.

Work Completed and evidence

Date	Commit time	Work Duration	Work completed and evidence	Driver	Navigator	Commit ID
16 October 2026	17:31	1 hour	takeTurn exception and Scanner bug fixes	Student 2	Student 1	345cebd
13 October 2026	15:03	2 days	Card denomination, validation and getter.	Student 2	Student 1	7ded3fb
16 October 2026	15:56	3 days	Player drawing, discarding and deck connections.	Student 2	Student 1	78c808d
13 October 2026	19:56	2 days	FIFO deck operations in CardDeck.	Student 1	Student 2	6db6ede
16 October 2026	18:00	3 days	CardGame thread coordination.	Student 1	Student 2	d6341ec

Reflection

During development, we encountered an input bug where the program displays input failure when the input is closed. The reason being when Scanner.nextLine() tried to read after input had closed. The fix we implemented was to add hasNextLine(), which checks for and handles the end of the input. Therefore the program can exit cleanly. We also discovered an issue where the program shows repetitive winners with the original pack. We therefore improved the card distribution of the pack, making sure the arrangement of the pack so each player gets a more equal chance of winning, and show more player activity. This change has improved the demonstration, but the outcome of the game will still depend on the scheduling of the pack.